Permutations.

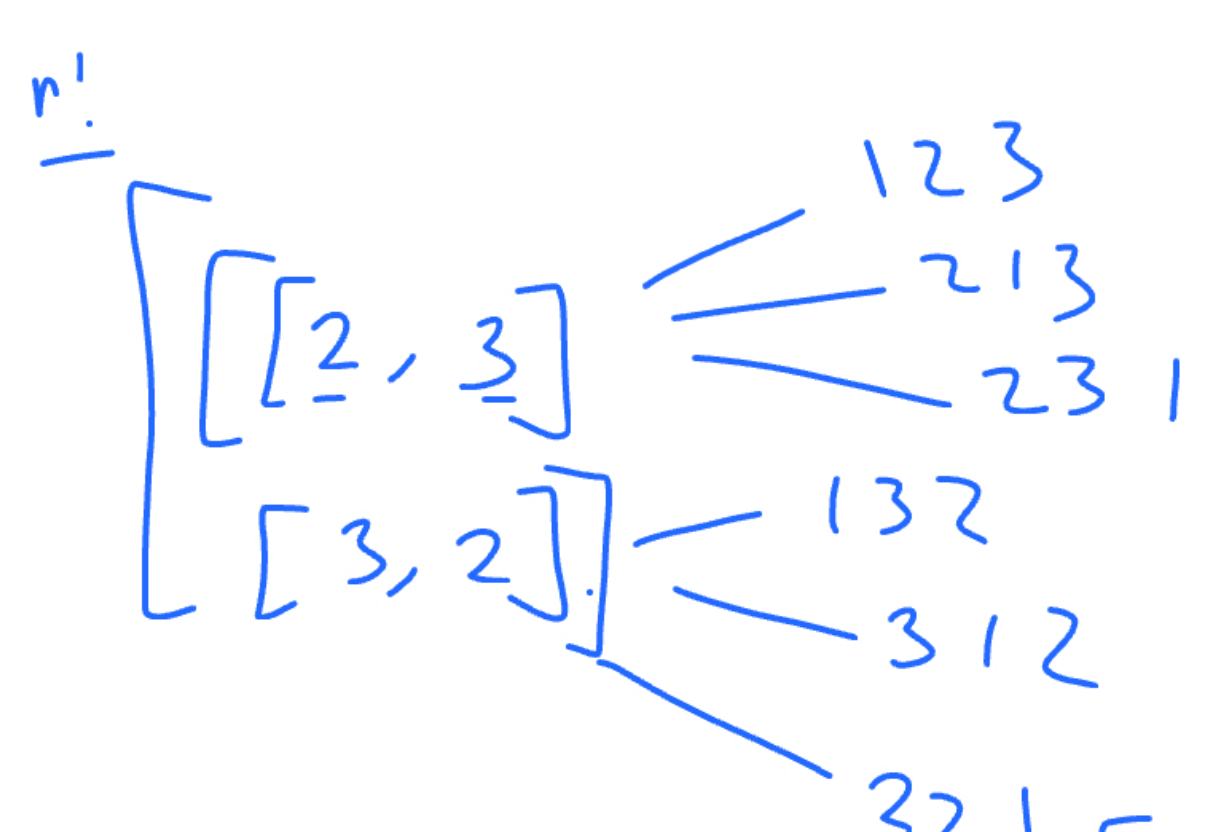
[1, 2, 3] — 123
132
213
231
312
321

n n-1 n-2 n-3 ... 1

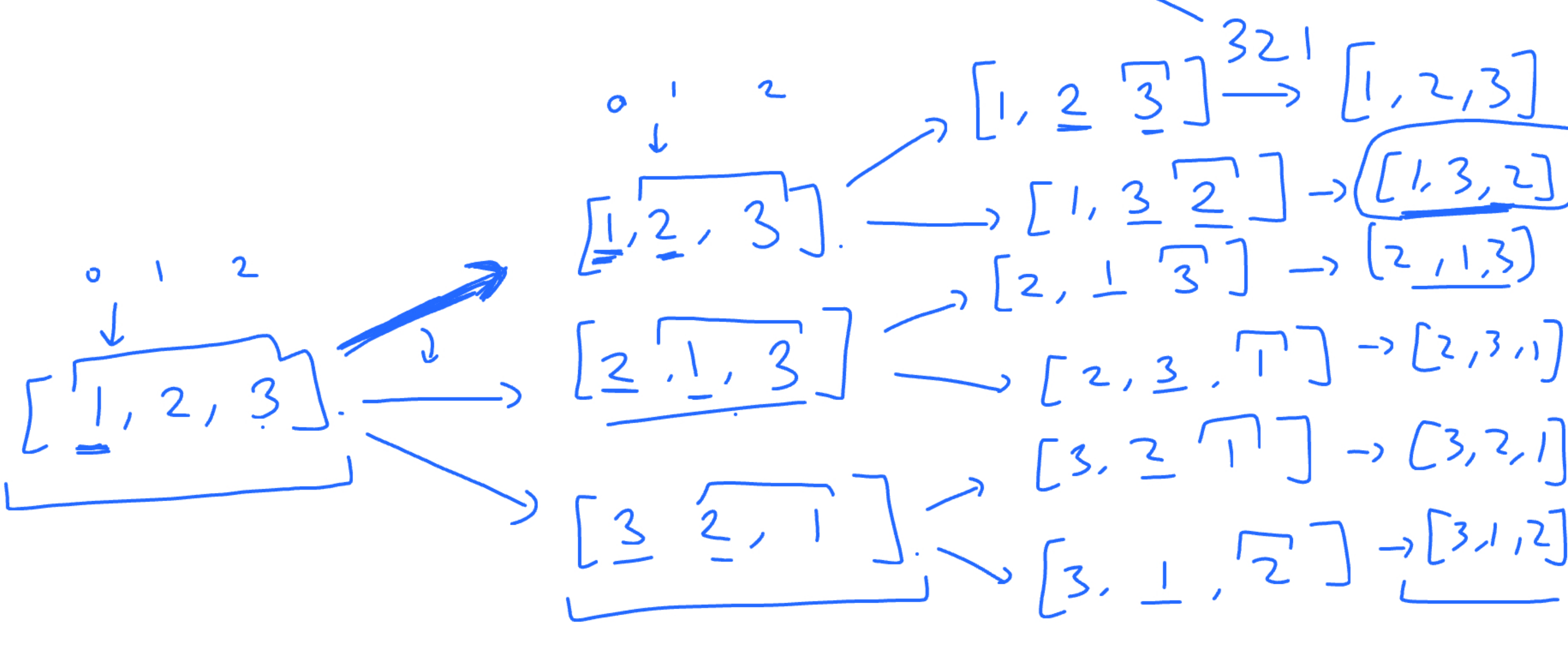
$n \times (n-1) \times (n-2) \cdots \times 1$

$= n!$

[1, 2, 3]



n!
[[2, 3]
[3, 2]]
123
213
231
132
312
321

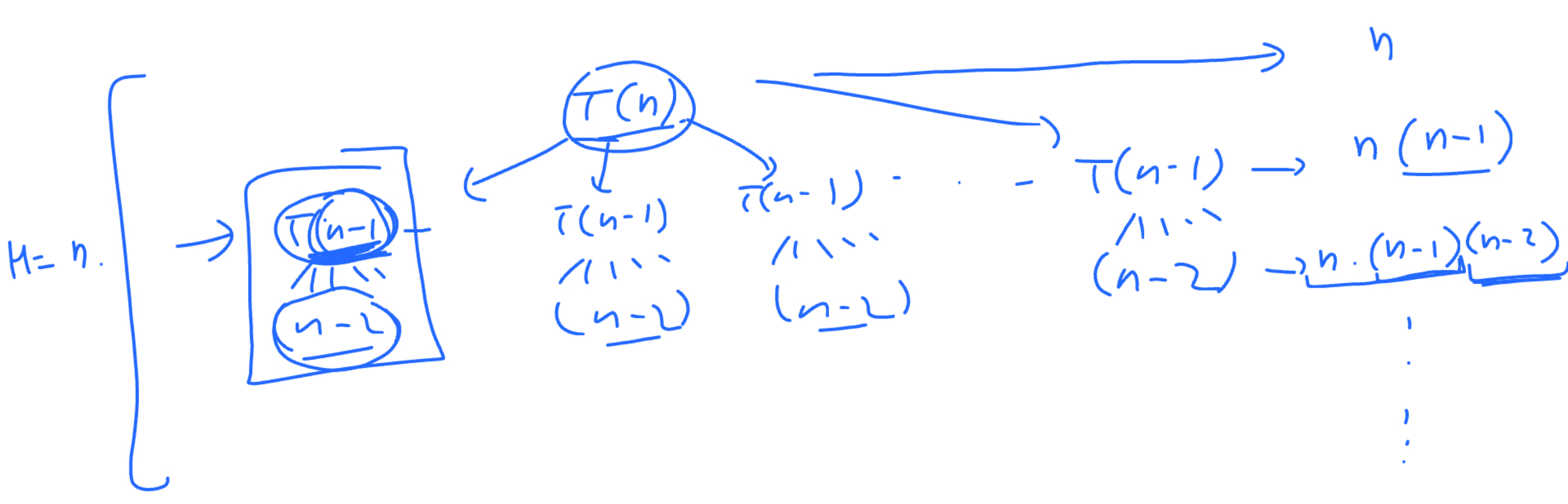


0 1 2
[1, 2, 3]
[1, 2, 3]
[2, 1, 3]
[3 2, 1]
[1, 2 3] → [1, 2, 3]
[1, 3 2] → [1, 3, 2]
[2, 1 3] → [2, 1, 3]
[2, 3, 1] → [2, 3, 1]
[3, 2 1] → [3, 2, 1]
[3, 1, 2] → [3, 1, 2]

i : 0 → n
i : 1 → n-1
i : 2 → n-2 ⇒ i : n - i

i	j
10	15
15	10
10	15

n!



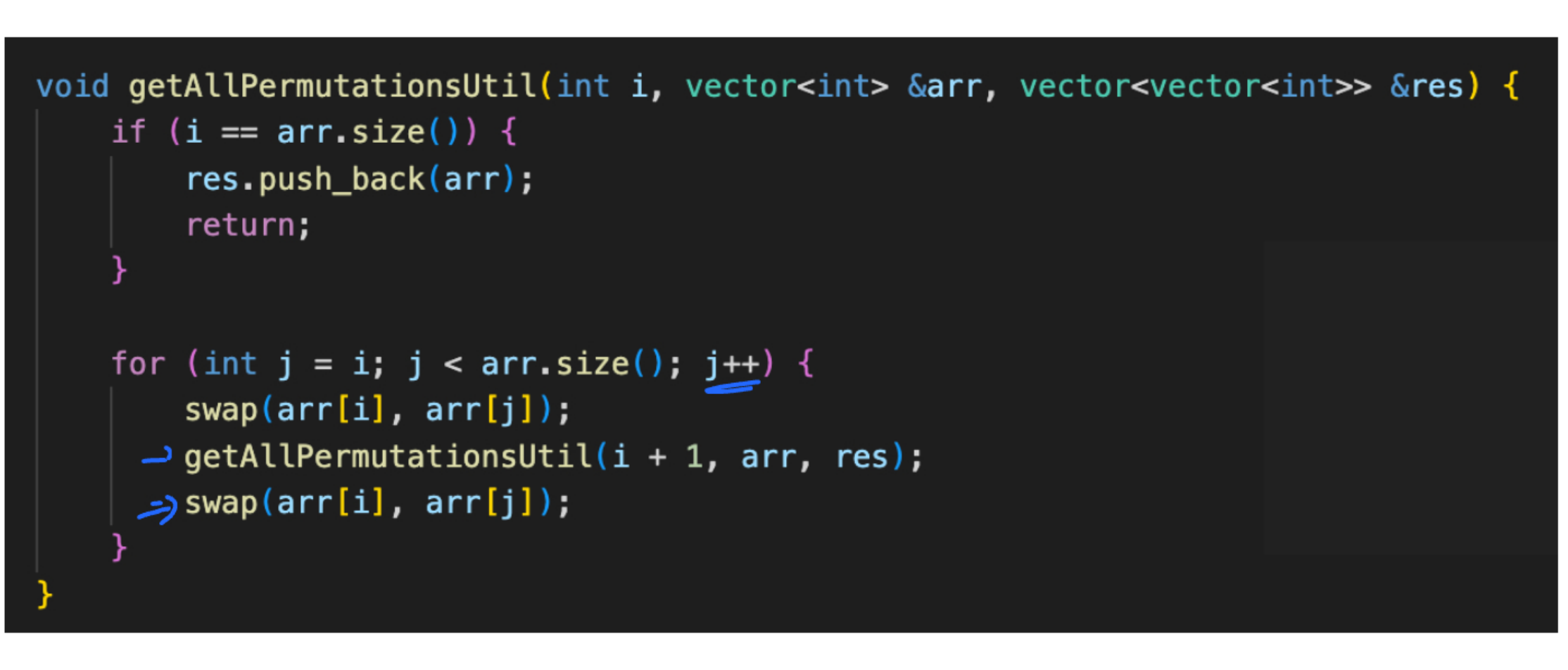
H = n.
T(n)
T(n-1)
(n-2)
T(n-1)
(n-2)
T(n-1) → n(n-1)
(n-2) → n.(n-1)(n-2)
n

$[\underline{n} + \underline{n(n-1)} + \underline{n(n-1)(n-2)} + n(n-1)(n-2)(n-3) \cdots + \underline{n!}]$

TC

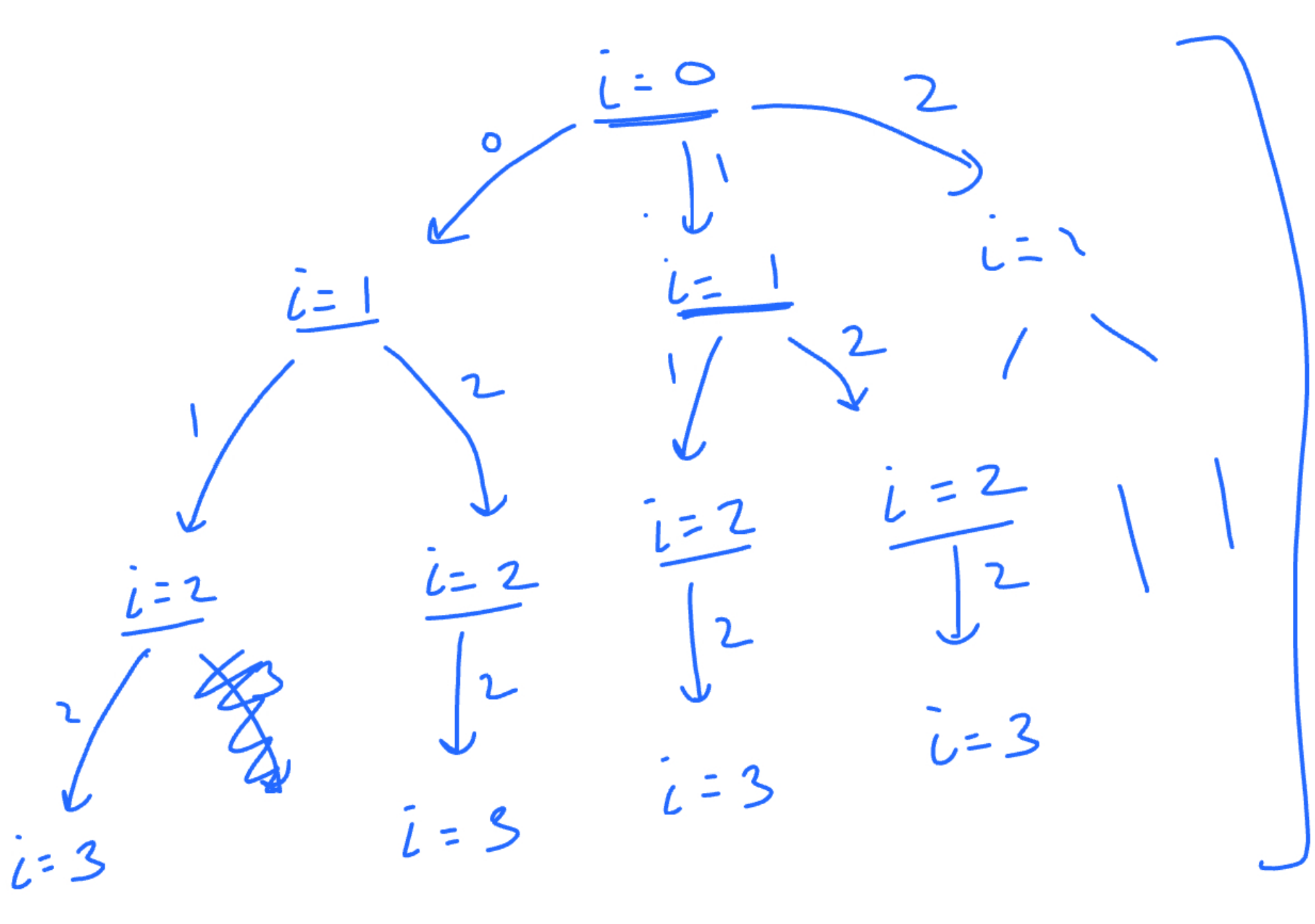
$\Rightarrow O(n \cdot n!)$

AS $\Rightarrow$ $O(n)$

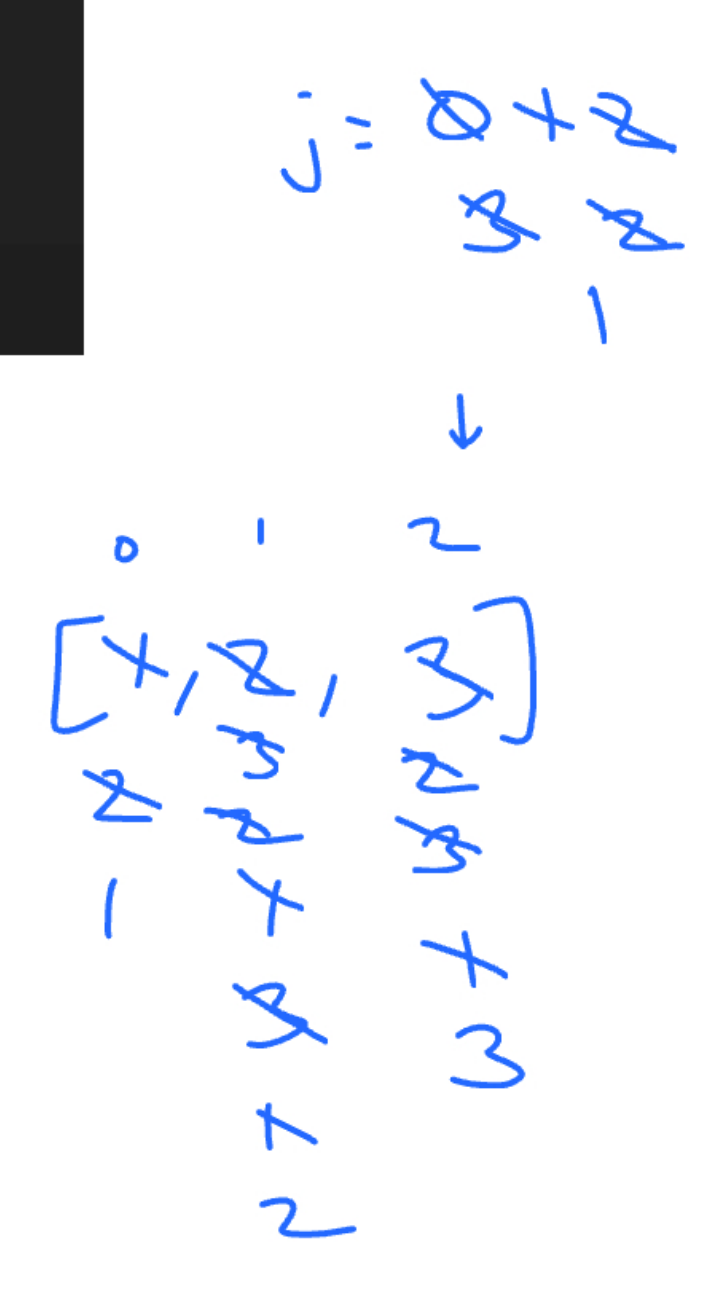


```
void getAllPermutationsUtil(int i, vector<int> &arr, vector<vector<int>> &res) {
    if (i == arr.size()) {
        res.push_back(arr);
        return;
    }

    for (int j = i; j < arr.size(); j++) {
        swap(arr[i], arr[j]);
        getAllPermutationsUtil(i + 1, arr, res);
        swap(arr[i], arr[j]);
    }
}
```



i=0
0
1
2
i=1
i=1
i=1
1
2
1
2
i=2
i=2
i=2
i=2
2
2
2
2
i=3
i=3
i=3
i=3

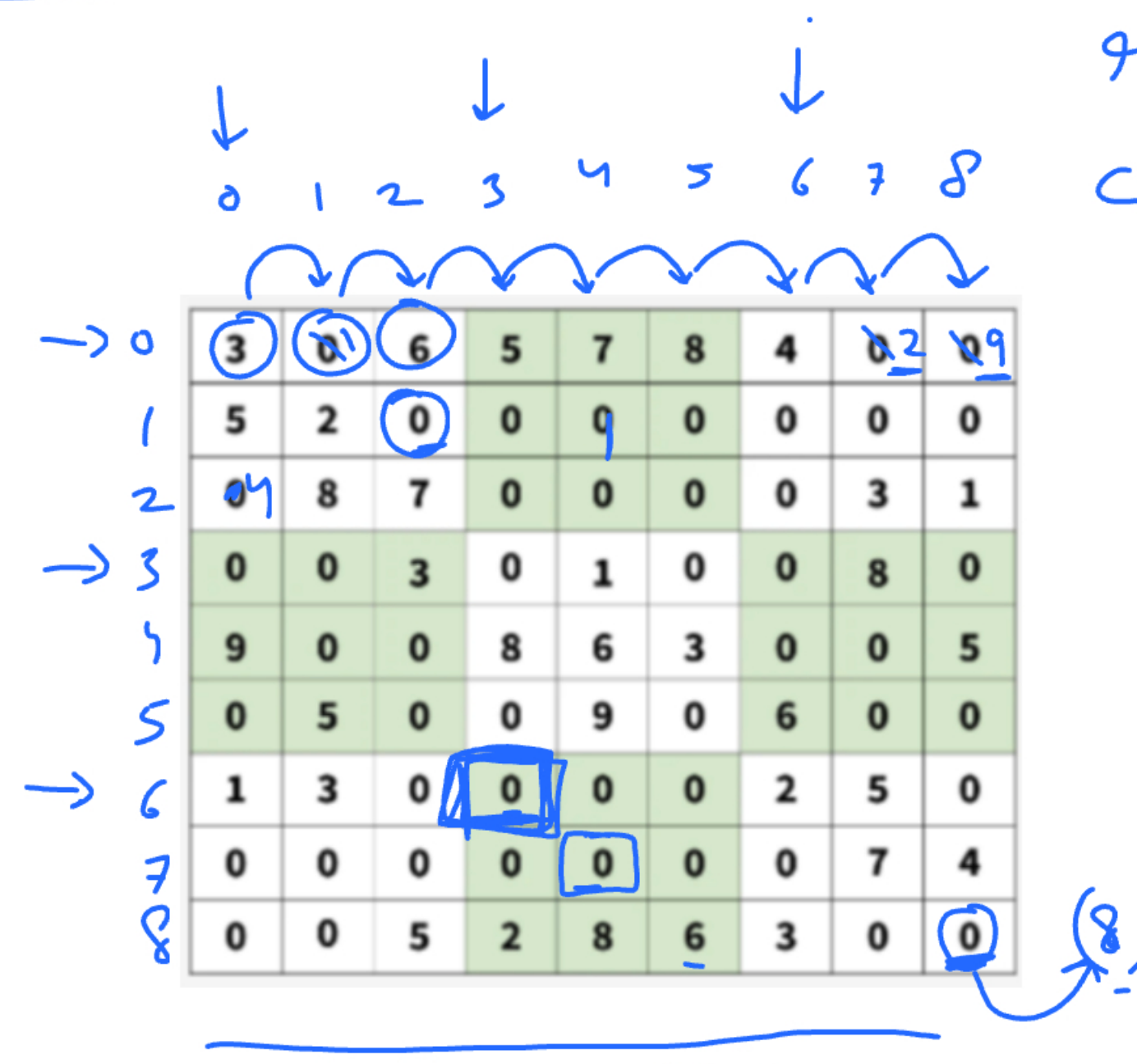


j = 0 1 2
1
0 1 2
[1, 2, 3]
2 3 2
1 1 3
3 1
2 3

[1, 2, 3].
[1, 3, 2].
[2, 1, 3]
[2, 3, 1].

---

i = 7



0 1 2 3 4 5 6 7 8
0
1
2
3
4
5
6
7
8
2
9
1
9
4
(8, 9)

3	0	6	5	7	8	4	0	0
5	2	0	0	0	0	0	0	0
0	8	7	0	0	0	0	3	1
0	0	3	0	1	0	0	8	0
9	0	0	8	6	3	0	0	5
0	5	0	0	9	0	6	0	0
1	3	0	0	0	0	2	5	0
0	0	0	0	0	0	0	7	4
0	0	5	2	8	6	3	0	0

$r = i - (i \% 3)$
$c = j - (j \% 3)$
i = 0 j = 8 → (1, 0).

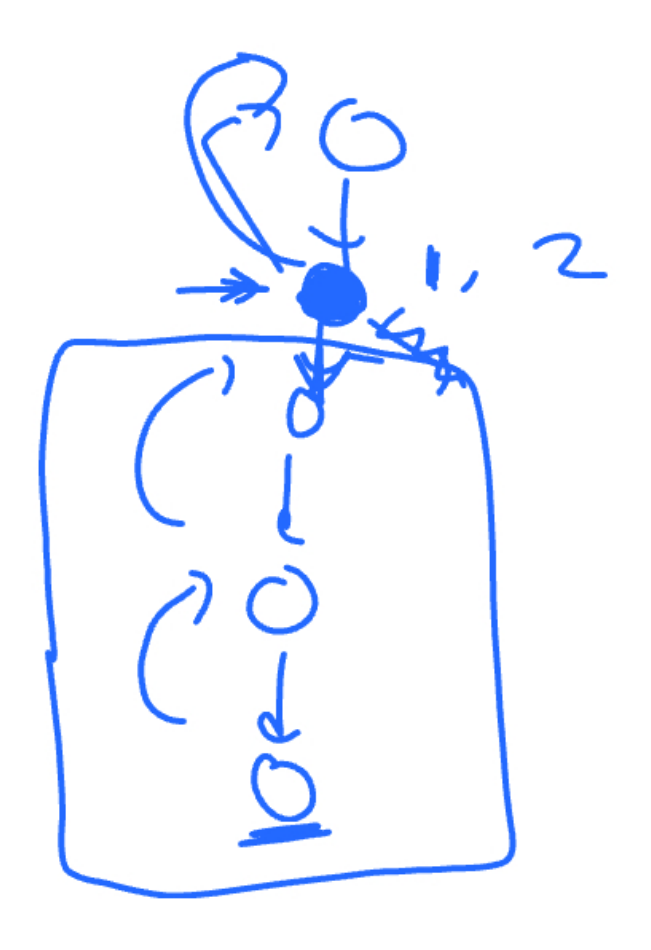
(i, j) → (i, j+1) ~~if j ≤ 8~~

~~(i, j) → (i+1, 0) if j > 8~~

(1, 3)
(8, 5)

[i: 8, j = 9]

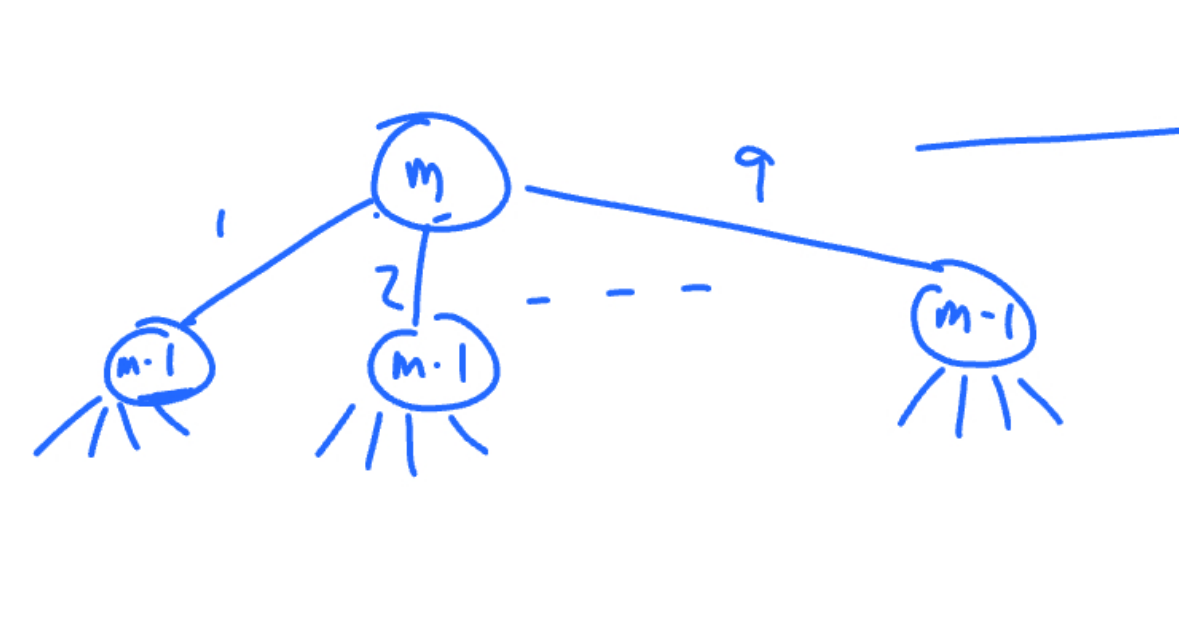
(i, 9) → (i+1, 0)



N = 9.

m

$= N^2$



m
1
2
9
m-1
m-1
m-1

→ 1
→ 9 → N
→ $9^2 \to N^2$
→ N^3

$\underbrace{1 + N + N^2 + N^3 \cdots}_{N^2 \text{ terms}}$

$1 \cdot \left(\frac{N^m - 1}{N - 1}\right) = O(9^m)$

where m is the no. of un filled cells.

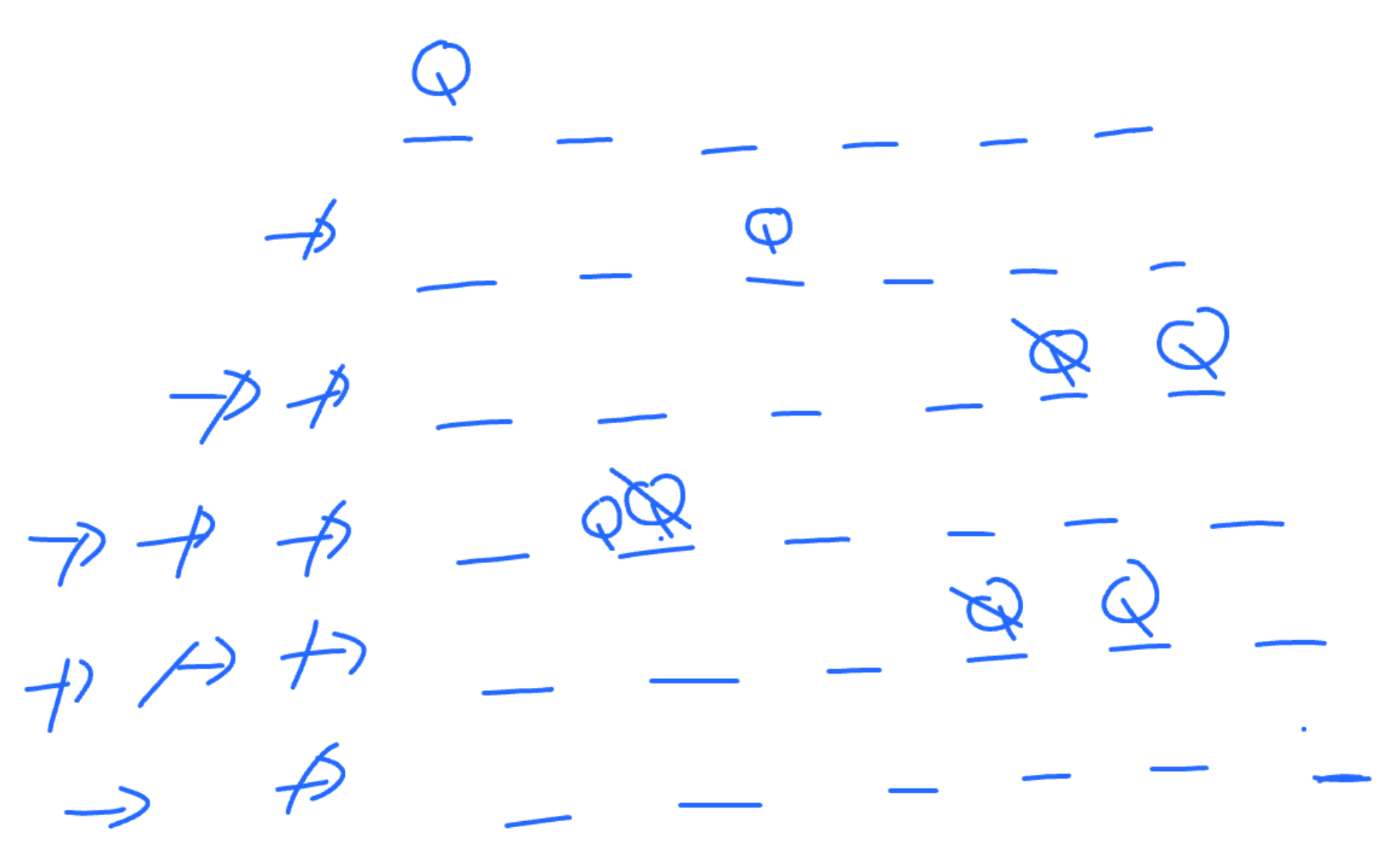
Q		
		Q

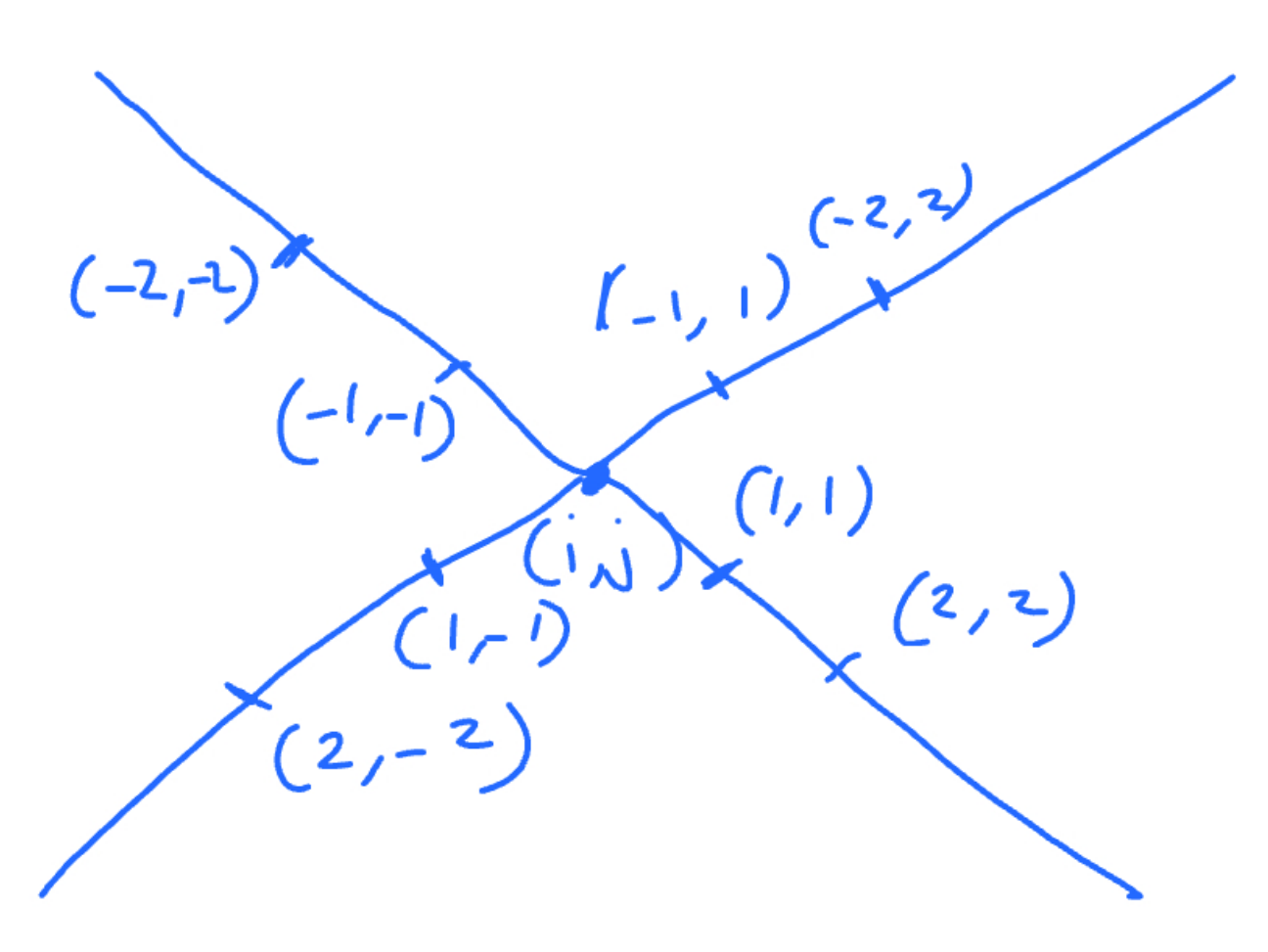
N = 4.

[" . Q . . "
. . . Q
Q . . .
. . Q .]

vector<vector<string>>.

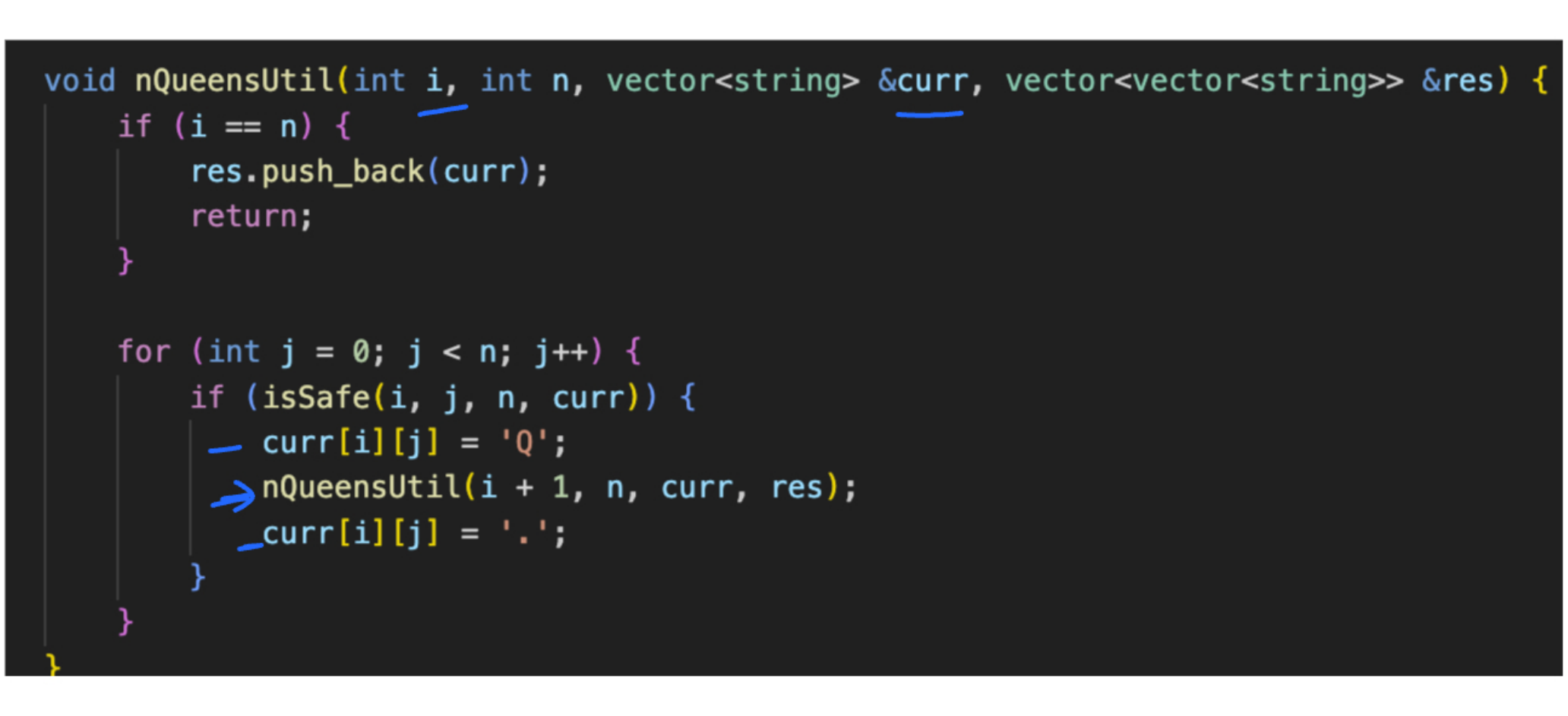
→ res.





(-2,-2)
(-1,-1)
(-1, 1)
(-2, 2)
(i, j)
(1, 1)
(2, 2)
(1, -1)
(2, -2)

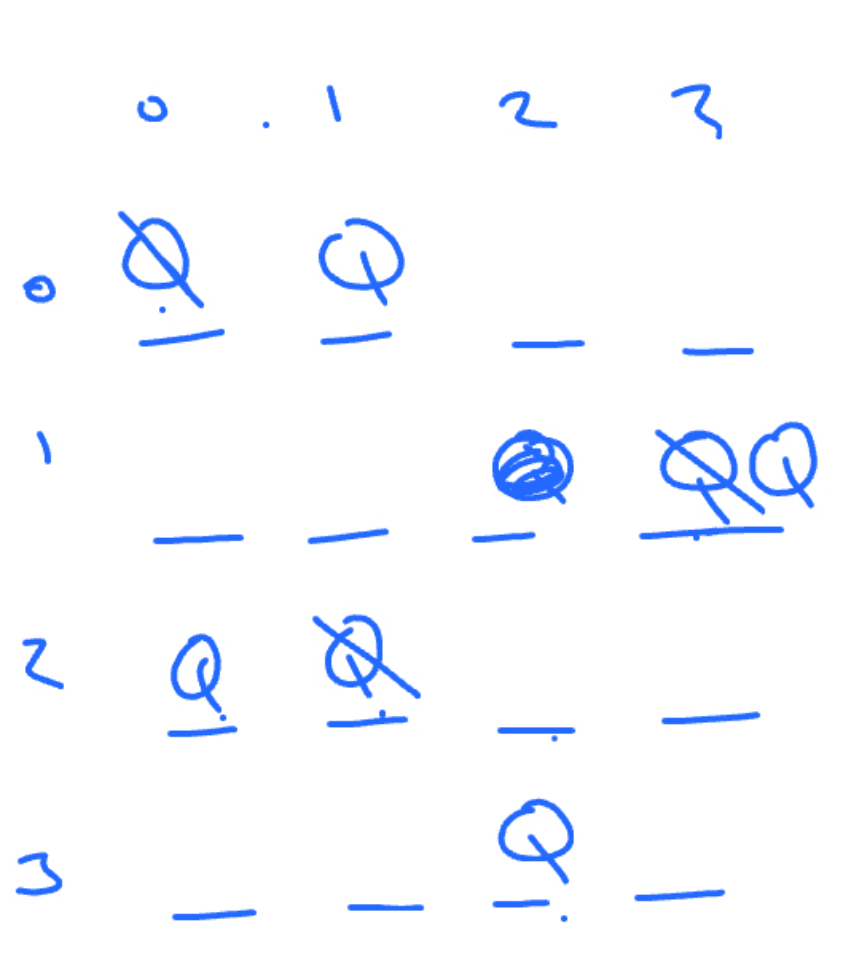
abs(r1 - r2) == abs (c1 - c2).
then
(r1, c1) & (r2, c2) lie on the same diagonal.



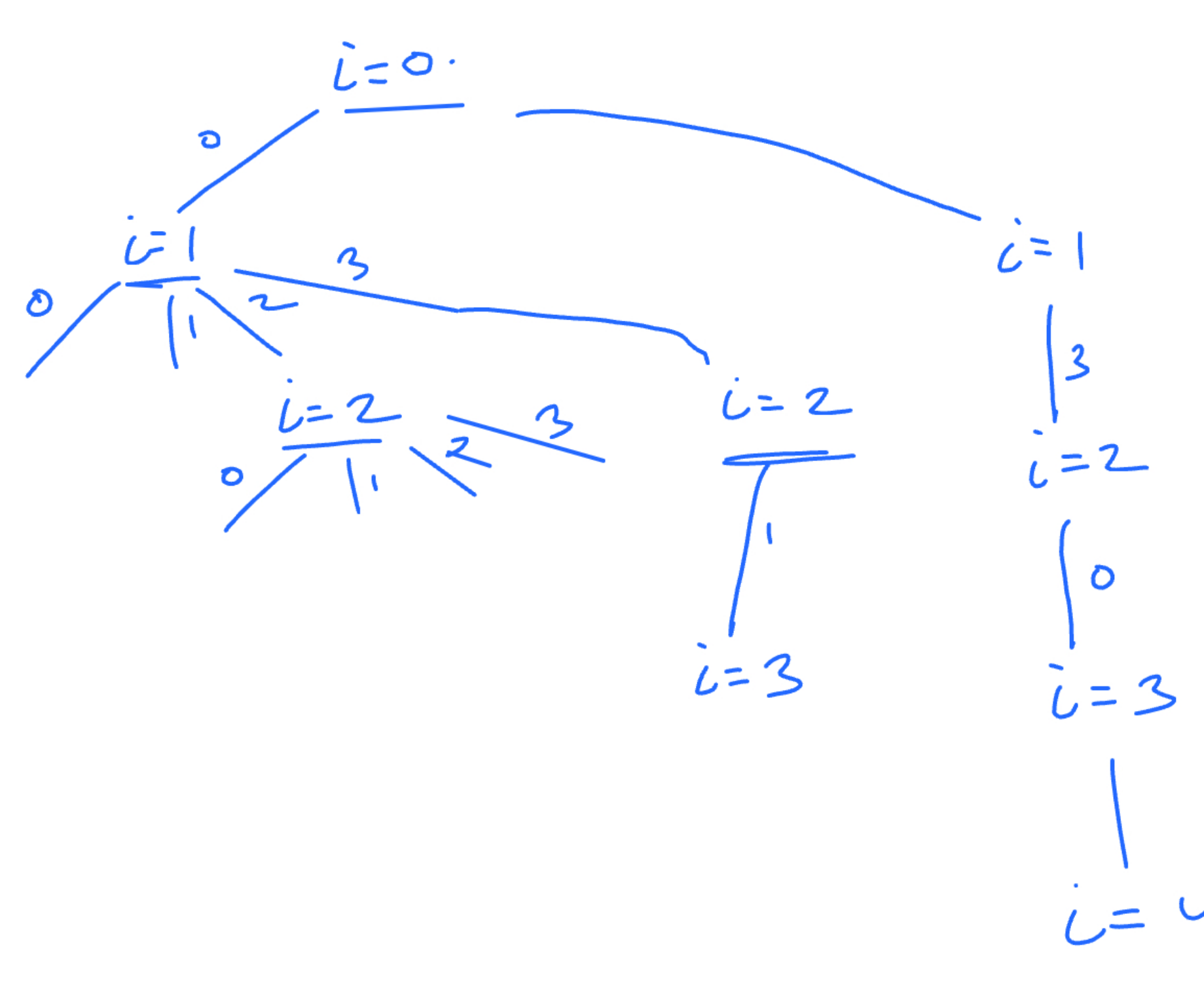
```
void nQueensUtil(int i, int n, vector<string> &curr, vector<vector<string>> &res) {
    if (i == n) {
        res.push_back(curr);
        return;
    }

    for (int j = 0; j < n; j++) {
        if (isSafe(i, j, n, curr)) {
            curr[i][j] = 'Q';
            nQueensUtil(i + 1, n, curr, res);
            curr[i][j] = '.';
        }
    }
}
```

n = 4



0 1 2 3
0
1
2
3
Q Q
Q QQ
Q Q
Q



i=0.
0
i=1
0
1
2
3
i=2
0
1
2
3
i=2
1
i=3
i=1
3
i=2
0
i=3
i=4

curr = int.

vector<int>

vector<string>.

vector<vector<string>>.